

AI Upskilling Platform

Cricket-Personalised Lessons v2 — Concept → Analogy → Code

CONTENTS

 Reading

Java Mastery: Introduction & Core Concepts

 Exercise

Python for Everyone: Hands-on Exercise

 Quiz

React 19 & Ecosystem: Knowledge Check

 Reading

System Design: Deep Dive & Advanced Patterns

 Project

AWS Solutions Architect: Capstone Mini Project

Interest:  Cricket · Pattern: concept →  analogy → code · 2026-05-29

Java Mastery: Introduction & Core Concepts

Topic: Java Mastery

Overview

Java is a statically typed, object-oriented programming language designed around the principle of "write once, run anywhere." The Java compiler (javac) does not produce native machine code; instead it produces an intermediate, platform-neutral format called bytecode, stored in .class files. This bytecode is executed by the Java Virtual Machine (JVM), an abstract computing machine implemented for each operating system. Because the JVM presents an identical execution environment regardless of the underlying hardware or OS, the same compiled program runs unchanged on Windows, Linux, or macOS. This platform independence, combined with strong typing, automatic memory management, and a vast standard library, is why Java dominates enterprise software: Spring Boot powers backend microservices, Android applications run on a JVM-derived runtime, and banking and trading systems rely on Java for stability, performance, and long-term maintainability.

🔪 Think of it like cricket: Java bytecode is like the Laws of Cricket — the same rules apply whether the match is played in Mumbai, Lord's, or Melbourne. The JVM is the umpire who enforces those same laws regardless of the local conditions (operating system). Just as a cricket match can be played on any ground that meets the standard pitch dimensions, a Java .class file runs on any JVM-compliant platform.

Core Concepts — OOP and the Type System

Object-oriented programming organises software around objects rather than functions and logic. A class is a blueprint that defines the state (fields) and behaviour (methods) shared by a category of things; an object is a concrete instance of that blueprint created at runtime. OOP rests on four pillars. Encapsulation bundles data with the methods that operate on it and hides internal state behind access modifiers. Inheritance lets a subclass acquire the fields and methods of a superclass, modelling an "is-a" relationship. Polymorphism allows a single reference type to refer to objects of different concrete types, dispatching method calls based on the actual object. Abstraction exposes essential behaviour while hiding implementation detail. Java's type system enforces these contracts at compile time, catching mismatches before the program runs. Java supports single inheritance of classes but permits a class to implement multiple interfaces, combining a strict hierarchy with flexible contracts.

✂ Think of it like cricket: A class is like the 'Batsman' role specification — it defines what a batsman does (face deliveries, score runs, protect wicket). Encapsulation means a batsman's technique is their own — you can see the runs scored (public method) but not the mental calculations behind shot selection (private fields). Inheritance is the AllRounder who IS-A Batsman AND IS-A Bowler. Polymorphism means when a Batsman and a Bowler both implement the CricketPlayer interface's play() method, each performs their role differently.

java

```
import java.util.Arrays;
import java.util.List;

abstract class CricketPlayer {
    protected String name;
    protected String teamName;
    protected int matchesPlayed;

    CricketPlayer(String name, String teamName, int matchesPlayed) {
        this.name = name;
        this.teamName = teamName;
        this.matchesPlayed = matchesPlayed;
    }

    abstract String getRole();
    abstract String getStatSummary();

    String introduce() {
        return name + " (" + teamName + ", " + matchesPlayed + " matches) - " + getRole();
    }
}

class Batsman extends CricketPlayer {
    private double battingAverage;
    private int totalRuns;

    Batsman(String name, String teamName, int matchesPlayed, double battingAverage, int
totalRuns) {
        super(name, teamName, matchesPlayed);
        this.battingAverage = battingAverage;
        this.totalRuns = totalRuns;
    }

    @Override
    String getRole() { return "Batsman"; }

    @Override
    String getStatSummary() {
        return "Avg " + battingAverage + ", Runs " + totalRuns;
    }
}
```

```

}

class Bowler extends CricketPlayer {
    private int wickets;
    private double economy;

    Bowler(String name, String teamName, int matchesPlayed, int wickets, double economy) {
        super(name, teamName, matchesPlayed);
        this.wickets = wickets;
        this.economy = economy;
    }

    @Override
    String getRole() { return "Bowler"; }

    @Override
    String getStatSummary() {
        return "Wickets " + wickets + ", Econ " + economy;
    }
}

interface Captain {
    String declareCaptain();
    void setFieldingPositions();
}

class AllRounder extends Batsman implements Captain {
    private int wickets;
    private double economy;

    AllRounder(String name, String teamName, int matchesPlayed,
        double battingAverage, int totalRuns, double economy) {
        super(name, teamName, matchesPlayed, battingAverage, totalRuns);
        this.economy = economy;
    }

    @Override
    String getRole() { return "All-Rounder"; }

    @Override
    public String declareCaptain() { return name + " leads " + teamName; }
}

```

```

@Override
public void setFieldingPositions() {
    System.out.println(name + " is setting the field.");
}
}

public class TeamLineup {
    public static void main(String[] args) {
        List<CricketPlayer> lineup = Arrays.asList(
            new Batsman("Rohit Sharma", "India", 200, 48.5, 9000),
            new Bowler("Jasprit Bumrah", "India", 150, 33, 2.7),
            new AllRounder("Ben Stokes", "England", 100, 35.2, 120, 3.5)
        );
        lineup.forEach(p → System.out.println(p.introduce() + " | " + p.getStatSummary()));
    }
}

```

The abstract class `CricketPlayer` cannot be instantiated directly — you can never field a generic "player," only a concrete `Batsman`, `Bowler`, or `AllRounder`. Each subclass supplies its own `getRole()` and `getStatSummary()`. When the loop calls `p.getStatSummary()` on a `CricketPlayer` reference, the JVM does not decide which method to run based on the reference type; it performs a virtual method table lookup on the actual runtime object, dispatching to `Batsman.getStatSummary()` for the batsman and `Bowler.getStatSummary()` for the bowler. The `Captain` interface decouples the captaincy contract from the player hierarchy, so any player type can become a captain without altering the inheritance chain.

How It Works Under the Hood

When you compile Java source, `javac` translates each `.java` file into a `.class` file containing bytecode. At runtime the JVM loads classes lazily through its `ClassLoader` hierarchy — a class is only loaded, linked, and initialised the first time it is actually needed, not all at once at startup. The bytecode initially runs through an interpreter, but the JVM's Just-In-Time (JIT) compiler — HotSpot's tiered C1 and C2 compilers — profiles execution and identifies "hot" methods that are invoked frequently (roughly after 10,000 invocations). These hot methods are compiled to optimised native machine code. Memory is split between the heap, where objects live, and per-thread stacks, which hold method call frames and local variables. The garbage collector — G1 by default in Java 17+ — automatically reclaims heap memory occupied by objects that are no longer reachable. Polymorphic calls are resolved through a virtual method table (vtable) maintained per class, enabling dynamic dispatch with minimal overhead.

 Think of it like cricket: The ClassLoader is the team selection committee — it brings players (classes) onto the field only when needed, not all at once. The JIT compiler is like a cricket team analyst who studies match footage (execution data) and tells players to refine their technique for the current pitch conditions (optimise hot code paths). The garbage collector is the groundskeeper who clears the boundary rope area (heap) once spectators (objects) have left after the match.

java

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;

class ScoreCard<T extends CricketPlayer> {
    private final List<T> players = new ArrayList<>();

    void addPlayer(T player) {
        players.add(player);
    }

    Optional<T> getTopPerformer() {
        return players.stream()
            .max(Comparator.comparingInt(p → p.matchesPlayed));
    }

    void sortByPerformance() {
        players.sort(Comparator.comparingInt((T p) → p.matchesPlayed).reversed());
    }
}

public class ScoreCardDemo {
    public static void main(String[] args) {
        ScoreCard<Batsman> batting = new ScoreCard<>();
        batting.addPlayer(new Batsman("Virat Kohli", "India", 250, 52.0, 12000));
        System.out.println(
            batting.getTopPerformer()
                .map(Batsman::getStatSummary)
                .orElse("No players")
        );
    }
}
```

💡 Pro Tip: When using generics with collections, prefer `List<? extends CricketPlayer>` for read-only use cases and `List<T extends CricketPlayer>` for write operations. Using raw types (`List` instead of `List<CricketPlayer>`) bypasses compile-time type checking and can cause `ClassCastException` at runtime — the kind of bug that crashes a live scoring system mid-match.

Common Patterns & Best Practices

The Builder pattern solves the problem of constructing complex objects with many parameters. A Match object needs two teams, a venue, a format, and over limits; a telescoping constructor with five or more parameters is error-prone, since callers can easily swap two same-typed arguments. A Builder provides a fluent, named API and validates required fields in `build()`, making invalid state impossible to construct. The Factory pattern decouples object creation from the calling code: a `PlayerFactory.create("batsman", stats)` call can return the correct subtype without the caller knowing the concrete class. The Strategy pattern encapsulates interchangeable algorithms behind a common interface, letting a Match apply different scoring rules — T20, Test, or ODI — by swapping a `ScoringStrategy` implementation rather than editing the Match class itself, keeping it closed for modification but open for extension.

✏️ Think of it like cricket: The Builder pattern is like the ICC's match approval process — you must specify playing conditions, venue, teams, and format before the match is confirmed (`build()` is called). The Factory is the player auction system — you request a 'fast bowler under 25' and the system returns the right player. The Strategy pattern is the format rules — the same Match object can play T20 (max 6 per over), Test (no over limit), or ODI (50 overs) by swapping the `ScoringStrategy`.

java

```
class Match {
    enum Format { T20, ODI, TEST }

    private final String team1;
    private final String team2;
    private final String venue;
    private final Format format;
    private final int overs;

    private Match(Builder b) {
        this.team1 = b.team1;
        this.team2 = b.team2;
        this.venue = b.venue;
        this.format = b.format;
        this.overs = b.overs;
    }

    static class Builder {
        private String team1;
        private String team2;
        private String venue;
        private Format format;
        private int overs;

        Builder withTeam1(String team1) { this.team1 = team1; return this; }
        Builder withTeam2(String team2) { this.team2 = team2; return this; }
        Builder withVenue(String venue) { this.venue = venue; return this; }

        Builder withFormat(Format format) {
            this.format = format;
            this.overs = (format == Format.T20) ? 20 : (format == Format.ODI) ? 50 : 0;
            return this;
        }

        Match build() {
            if (team1 == null || team2 == null || venue == null || format == null) {
                throw new IllegalStateException("Match requires both teams, a venue, and a
format.");
            }
            return new Match(this);
        }
    }
}
```

```

    }
}

public static void main(String[] args) {
    Match ipl = new Match.Builder()
        .withTeam1("Mumbai Indians")
        .withTeam2("CSK")
        .withVenue("Wankhede")
        .withFormat(Format.T20)
        .build();

    System.out.println("Match created: " + ipl.team1 + " vs " + ipl.team2 + " at " +
        ipl.venue);
}
}

```

 **Warning:** Never compare Integer objects with `==` in Java — this works for values -128 to 127 (cached by the JVM) but fails for larger numbers. A cricket scoreboard comparing `playerScore == targetScore` will work for small scores but silently return false for centuries. Always use `.equals()` for object comparison: `playerScore.equals(targetScore)`.

Real-World Application

Java powers Cricinfo's backend APIs and Dream11's fantasy platform, which handles over 100 million users during the IPL using Spring Boot microservices. Every Android cricket app runs on the JVM-derived runtime (with Kotlin compiling to the same bytecode). HDFC Bank, Goldman Sachs, and Amazon all run core systems on Java for its stability and ecosystem. Java 21's virtual threads (Project Loom) are a major leap for such workloads — they let a server handle millions of concurrent score-update connections without the heavy memory cost of the traditional thread-per-request model, ideal for live cricket streaming at scale.

KEY POINTS

- JVM vtable dispatch enables polymorphism with zero conditional logic in the calling code, resolving the right method at runtime.
- Generics with bounded wildcards prevent `ClassCastException` at compile time rather than blowing up at runtime.
- The Builder pattern with validation in `build()` makes invalid object state impossible to construct in the first place.
- The JIT C2 compiler inlines virtual method calls after profiling, eliminating polymorphism overhead in hot code paths.
- Java's checked exceptions force callers to handle error cases, unlike unchecked exceptions which can silently propagate.
- G1GC's region-based collection minimises stop-the-world pauses, which is critical for latency-sensitive applications.

Q: A `CricketPlayer` variable holds a `Batsman` object at runtime. You call `player.getStatSummary()`. Which Java mechanism determines that `Batsman`'s version of `getStatSummary()` is called?

- A** Static binding at compile time based on variable type
- B** Dynamic dispatch via the vtable at runtime based on actual object type
- C** Reflection API at runtime
- D** The `instanceof` operator

Explanation: Java uses dynamic dispatch (runtime polymorphism) for non-static, non-private methods. The JVM maintains a virtual method table (vtable) per class. When `getStatSummary()` is called on a `CricketPlayer` reference, the JVM looks up the vtable of the actual runtime object type (`Batsman`) and dispatches to `Batsman.getStatSummary()`, not `CricketPlayer`'s version.

Q: Your cricket app stores match scores as Integer objects in a Map. A unit test checks if `score == 150` (Integer comparison). It passes for scores 0-127 but fails for scores above 127. What is the root cause?

A Integer overflow above 127

B JVM Integer cache only covers -128 to 127; above that, `==` compares references not values

C `Map.get()` returns null for scores above 127

D Integer is not comparable above 127

Explanation: Java caches Integer objects from -128 to 127 in a pool. Within this range, `Integer.valueOf(100)` always returns the same object, so `==` works. Above 127, each `Integer.valueOf(150)` creates a new object, so `==` compares memory addresses (always false for different instances). Use `.equals()` for all Integer comparisons.

Python for Everyone: Hands-on Exercise

Topic: Python for Everyone

What You'll Build

You'll build a CLI cricket scorecard tracker — a Python application that records batting innings (batsman, runs, balls faced, fours, sixes, dismissal method), bowling figures (bowler, overs, maidens, runs conceded, wickets), calculates strike rate, economy rate, and run rate, and persists all match data to JSON files. Each command runs as a CLI subcommand. This is a perfect project for learning Python dataclasses, JSON I/O, argparse, and formatted output. By the end you'll have a working tool you can run from the terminal to score a real match ball by ball, with each match stored in its own JSON file for later retrieval and display.

🔪 Think of it like cricket: This app is exactly what a cricket scorer does during a live match — recording each batsman's innings on the physical scoresheet, updating bowling figures over-by-over, and calculating run rates. You're digitising the traditional paper scorecard into a JSON-backed Python program.

Prerequisites

KEY POINTS

- Python 3.8+ installed (python3 --version to check)
- Understanding of Python dicts and lists
- Basic knowledge of functions and modules
- Familiarity with running Python scripts from the terminal
- pip installed for any optional dependencies

Setup & Project Structure

The project uses only the Python standard library — no pip installs are needed. Create a single scorecard.py file and a data/ directory where match JSON files are saved. Each match gets its own JSON file named after the match (for example, data/india_vs_australia.json). Keeping data alongside

the script in a dedicated folder keeps the project self-contained and easy to inspect or version-control.

```
bash
```

```
mkdir cricket_scorecard && cd cricket_scorecard && mkdir data && touch scorecard.py
```

```
# Resulting project structure:
```

```
# cricket_scorecard/
```

```
# └─ scorecard.py
```

```
# └─ data/          (empty – JSON files are created here, one per match)
```

Step 1 — Foundation: Data Models

Python dataclasses provide a clean way to model structured data without boilerplate. We define `BattingEntry` and `BowlingEntry` as dataclasses, and an `Innings` class that holds both. The `@dataclass` decorator auto-generates `__init__`, `__repr__`, and `__eq__` from the field definitions, eliminating constructor boilerplate. JSON serialisation uses `dataclasses.asdict()` to convert each instance to a dict for `json.dump()`, and unpacking a dict back into the constructor (`**data`) reconstructs the objects on load.

 Think of it like cricket: A dataclass is like a scorecard row template — it defines exactly what columns exist for a batting entry (batsman, runs, balls, 4s, 6s, method of dismissal). Every batsman's row follows the same structure. The `Innings` dataclass is the full scorecard — a container holding all the batting rows and bowling rows for one team's innings.

python

```
from dataclasses import dataclass, field, asdict
from typing import List
import json, os

@dataclass
class BattingEntry:
    batsman: str
    runs: int = 0
    balls: int = 0
    fours: int = 0
    sixes: int = 0
    dismissal: str = "not out"

    def strike_rate(self) → float:
        return round((self.runs / self.balls) * 100, 2) if self.balls > 0 else 0.0

@dataclass
class BowlingEntry:
    bowler: str
    overs: float = 0.0
    maidens: int = 0
    runs_conceded: int = 0
    wickets: int = 0

    def economy(self) → float:
        return round(self.runs_conceded / self.overs, 2) if self.overs > 0 else 0.0

@dataclass
class Innings:
    team: str
    batting: List[BattingEntry] = field(default_factory=list)
    bowling: List[BowlingEntry] = field(default_factory=list)

    def total_runs(self) → int:
        return sum(b.runs for b in self.batting)

    def total_wickets(self) → int:
        return sum(bw.wickets for bw in self.bowling)

    def run_rate(self, overs: float) → float:
```



```

        return round(self.total_runs() / overs, 2) if overs > 0 else 0.0

SAVE_DIR = "data"

def save_innings(match_name: str, innings: Innings):
    os.makedirs(SAVE_DIR, exist_ok=True)
    path = os.path.join(SAVE_DIR, f"{match_name}.json")
    data = {
        "team": innings.team,
        "batting": [asdict(b) for b in innings.batting],
        "bowling": [asdict(bw) for bw in innings.bowling],
    }
    with open(path, "w") as f:
        json.dump(data, f, indent=2)

def load_innings(match_name: str) → Innings:
    path = os.path.join(SAVE_DIR, f"{match_name}.json")
    with open(path) as f:
        data = json.load(f)
    innings = Innings(team=data["team"])
    innings.batting = [BattingEntry(**b) for b in data["batting"]]
    innings.bowling = [BowlingEntry(**bw) for bw in data["bowling"]]
    return innings

```

Step 2 — Core Logic: Scorecard Operations

We add functions to update the innings: `add_batting_entry()` appends a `BattingEntry` and `add_bowling_entry()` appends a `BowlingEntry`, each loading the current innings, mutating it, and saving again. The `format_scorecard()` function produces a formatted string for display — using f-strings and field width specifiers for columnar alignment, the same way a printed scorecard aligns columns so totals and rates line up neatly.

 Think of it like cricket: Adding a batting entry is like the scorer recording a dismissal on the paper scorecard — once Rohit Sharma is out for 67, that row is finalised. The `format_scorecard()` function is the printer that produces the final printed scorecard you see on the ground scoreboard.

python

```
def add_batting_entry(match_name: str, batsman: str, runs: int, balls: int,
                      fours: int, sixes: int, dismissal: str):
    innings = load_innings(match_name)
    innings.batting.append(BattingEntry(batsman=batsman, runs=runs, balls=balls,
                                         fours=fours, sixes=sixes, dismissal=dismissal))
    save_innings(match_name, innings)

def add_bowling_entry(match_name: str, bowler: str, overs: float, maidens: int,
                      runs: int, wickets: int):
    innings = load_innings(match_name)
    innings.bowling.append(BowlingEntry(bowler=bowler, overs=overs, maidens=maidens,
                                         runs_conceded=runs, wickets=wickets))
    save_innings(match_name, innings)

def format_scorecard(match_name: str, overs_bowled: float = 20.0) → str:
    inn = load_innings(match_name)
    lines = [f"\n{'='*60}", f" {inn.team.upper()} INNINGS", f"{'='*60}"]
    lines.append(f" {'BATSMAN':<22} {'R':>4} {'B':>4} {'4s':>4} {'6s':>4} {'SR':>7}{'DISMISSAL'}")
    lines.append(f" {'-'*56}")
    for b in inn.batting:
        lines.append(f" {b.batsman:<22} {b.runs:>4} {b.balls:>4} {b.fours:>4} "
                    f"{b.sixes:>4} {b.strike_rate():>7.1f} {b.dismissal}")
    lines.append(f" {'-'*56}")
    lines.append(f" {'TOTAL':<22} {inn.total_runs():>4} "
                f"({inn.total_wickets()} wkts, {overs_bowled} overs) RR: {inn.run_rate(overs_bowled)}")
    lines.append(f"\n {'BOWLER':<22} {'O':>4} {'M':>4} {'R':>4} {'W':>4} {'ECON':>6}")
    lines.append(f" {'-'*50}")
    for bw in inn.bowling:
        lines.append(f" {bw.bowler:<22} {bw.overs:>4.1f} {bw.maidens:>4} {bw.runs_conceded:>4} "
                    f"{bw.wickets:>4} {bw.economy():>6.2f}")
    lines.append(f"{'='*60}\n")
    return "\n".join(lines)
```

Step 3 — Integration: CLI with argparse

argparse provides the CLI interface. Subcommands (new-match, add-bat, add-bowl, show) are registered as subparsers, and each subcommand has its own argument set with its own required and optional flags. The main() function inspects args.command and dispatches to the right handler. This is the exact pattern used by mature CLI tools like git, docker, and aws — one entry point that branches into many focused sub-actions.

 Think of it like cricket: argparse is like the scorer's command interface. 'new-match' is equivalent to opening a fresh scorebook for a new game. 'add-bat' is recording a batsman's dismissal. 'show' is reading out the current scorecard to the broadcast team. Each subcommand is a specific scorer action.

python

```
import argparse

def main():
    parser = argparse.ArgumentParser(description="Cricket Scorecard Tracker")
    sub = parser.add_subparsers(dest="command")

    # new-match
    nm = sub.add_parser("new-match", help="Start a new match scorecard")
    nm.add_argument("--match", required=True, help="Match name (used as filename)")
    nm.add_argument("--team", required=True, help="Batting team name")

    # add-bat
    ab = sub.add_parser("add-bat", help="Record a batting entry")
    ab.add_argument("--match", required=True)
    ab.add_argument("--batsman", required=True)
    ab.add_argument("--runs", type=int, required=True)
    ab.add_argument("--balls", type=int, required=True)
    ab.add_argument("--fours", type=int, default=0)
    ab.add_argument("--sixes", type=int, default=0)
    ab.add_argument("--dismissal", default="not out")

    # add-bowl
    aw = sub.add_parser("add-bowl", help="Record bowling figures")
    aw.add_argument("--match", required=True)
    aw.add_argument("--bowler", required=True)
    aw.add_argument("--overs", type=float, required=True)
    aw.add_argument("--maidens", type=int, default=0)
    aw.add_argument("--runs", type=int, required=True)
    aw.add_argument("--wickets", type=int, required=True)

    # show
    sh = sub.add_parser("show", help="Display the scorecard")
    sh.add_argument("--match", required=True)
    sh.add_argument("--overs", type=float, default=20.0)

    args = parser.parse_args()

    if args.command == "new-match":
        save_innings(args.match, Innings(team=args.team))
        print(f"New scorecard created: {args.match} ({args.team})")
```

```

elif args.command == "add-bat":
    add_batting_entry(args.match, args.batsman, args.runs, args.balls,
                      args.fours, args.sixes, args.dismissal)
    print(f"Added: {args.batsman} {args.runs}({args.balls})")
elif args.command == "add-bowl":
    add_bowling_entry(args.match, args.bowler, args.overs, args.maidens,
                      args.runs, args.wickets)
    print(f"Added bowling: {args.bowler} {args.wickets}/{args.runs} in {args.overs}
overs")
elif args.command == "show":
    print(format_scorecard(args.match, args.overs))
else:
    parser.print_help()

if __name__ == "__main__":
    main()

```

Step 4 — Testing & Verification

Run the commands below to simulate India's innings from the 2023 World Cup final. After entering all batting and bowling figures, the show command prints a formatted scorecard so you can verify that totals, strike rates, and run rate are all computed correctly.

```
bash
```

```
# Create new match
```

```
python3 scorecard.py new-match --match ind_vs_aus_final --team India
```

```
# Record India's batting
```

```
python3 scorecard.py add-bat --match ind_vs_aus_final --batsman "Rohit Sharma" --runs 47 --balls 31 --fours 5 --sixes 2 --dismissal "caught Labuschagne b Hazlewood"
```

```
python3 scorecard.py add-bat --match ind_vs_aus_final --batsman "Shubman Gill" --runs 4 --balls 7 --fours 1 --sixes 0 --dismissal "lbw b Hazlewood"
```

```
python3 scorecard.py add-bat --match ind_vs_aus_final --batsman "Virat Kohli" --runs 54 --balls 63 --fours 6 --sixes 0 --dismissal "caught Inglis b Cummins"
```

```
# Record Australia's bowling
```

```
python3 scorecard.py add-bowl --match ind_vs_aus_final --bowler "Pat Cummins" --overs 10.0 --maidens 2 --runs 34 --wickets 2
```

```
python3 scorecard.py add-bowl --match ind_vs_aus_final --bowler "Josh Hazlewood" --overs 10.0 --maidens 1 --runs 60 --wickets 4
```

```
# Show scorecard
```

```
python3 scorecard.py show --match ind_vs_aus_final --overs 50.0
```

```
# Expected output:
```

```
# =====
```

```
# INDIA INNINGS
```

```
# =====
```

```
# BATSMAN R B 4s 6s SR DISMISSAL
```

```
# -----
```

```
# Rohit Sharma 47 31 5 2 151.6 caught Labuschagne b Hazlewood
```

```
# Shubman Gill 4 7 1 0 57.1 lbw b Hazlewood
```

```
# Virat Kohli 54 63 6 0 85.7 caught Inglis b Cummins
```

```
# -----
```

```
# TOTAL 105 (3 wkts, 50.0 overs) RR: 2.1
```

 **Warning:** If you get 'FileNotFoundError' when running add-bat, it means the match file doesn't exist yet — you forgot to run new-match first, or spelled the match name differently. Match names are case-sensitive and used directly as filenames. Always create the match with new-match before adding any entries.

💡 **Extension Challenge:** Add live run-rate tracking — store the current over number in the JSON and auto-calculate the required run rate for the second innings. For extra depth, integrate with the cricinfo live score API (unofficial) using requests to auto-populate the scorecard from a real match URL.

KEY POINTS

- Python dataclasses auto-generate `__init__`, `__repr__`, and `__eq__`, eliminating boilerplate for structured data models.
- `dataclasses.asdict()` provides clean dict conversion for JSON serialisation without manual mapping.
- The `argparse` subparsers pattern matches the `git/docker` CLI convention — one tool, many subcommands with independent argument sets.
- f-strings with format specifiers (`:%<22, :>4`) enable clean columnar output without external libraries.
- `os.makedirs(exist_ok=True)` creates parent directories safely without first checking if they exist.
- Separating data model (dataclasses), business logic (add/format functions), and CLI (`argparse`) keeps code testable and maintainable.

React 19 & Ecosystem: Knowledge Check

Topic: React 19 & Ecosystem

Knowledge Check — React 19 & Ecosystem

This quiz tests your understanding of React hooks, rendering behaviour, performance optimisation, and component architecture. The applied questions (5-6) are framed as real engineering decisions you'd make building a live cricket scoring dashboard with real-time ball-by-ball updates, so you can connect each concept to a concrete production scenario.

Q: What is the primary purpose of React's `useCallback` hook?

- A It memoizes the result of a computation to avoid re-running on every render
- B It returns a memoized function reference so child components receiving it as a prop don't re-render unnecessarily**
- C It subscribes a component to a context value and re-renders on changes
- D It delays function execution until after the browser has painted

Explanation: `useCallback` memoizes the function reference itself, not its return value (that's `useMemo`). Without `useCallback`, a new function object is created on every render. Child components wrapped in `React.memo` that receive this function as a prop would see a new reference each render and re-render unnecessarily, defeating the purpose of memoization.

Q: In React 18+, you call `setScore(score + 1)` and `setWickets(wickets + 1)` inside a click handler. How many re-renders occur?

- A Two re-renders — one for each `setState` call
- B One re-render — React 18 automatically batches multiple `setState` calls in event handlers**
- C Zero re-renders until the next tick
- D It depends on whether the component is a class or function component

Explanation: React 18 introduced automatic batching for all state updates, including those in `setTimeout`, Promises, and native event handlers. Previously only React event handlers were batched. Both `setState` calls are collected and applied in a single re-render, improving performance.

Q: Your cricket scoreboard has a MatchHeader component wrapped in React.memo. You pass matchTitle='India vs Australia' (same string) and onRefresh={handleRefresh} (a function defined inline) as props. Does MatchHeader re-render when the parent re-renders?

A No, because both props are the same values

B Yes, because the inline function creates a new reference on every parent render, and React.memo uses reference equality for functions

C No, because React.memo deep-compares all props by default

D Yes, but only if matchTitle also changes

Explanation: React.memo uses shallow (reference) equality for prop comparison. Primitive values like strings are compared by value. But functions are objects — an inline arrow function creates a new object reference on every render. Even if the function body is identical, `{}` `!==` `{}` in JavaScript. Wrap `handleRefresh` in `useCallback` to stabilise its reference.

Q: In your live score app, `useEffect(() => { subscribe(matchId) }, [matchId])` is called. The `matchId` changes from 'match-1' to 'match-2'. In what order do the effects run?

A New effect for match-2 runs first, then cleanup for match-1

B Cleanup for match-1 runs first, then new effect for match-2

C Both run simultaneously in a microtask queue

D The cleanup runs only when the component unmounts, not on dependency change

Explanation: When a `useEffect` dependency changes, React runs the cleanup function of the previous effect before running the new effect. This prevents subscribing to match-2 while still subscribed to match-1. The cleanup is guaranteed to run before the next effect of the same `useEffect`.

Q: Your cricket dashboard renders an 11-player lineup. Each `BatsmanCard` shows the player's live strike rate. Every ball delivery updates one player's score, but all 11 `BatsmanCard` components re-render. What is the most correct fix?

- A** Wrap `BatsmanCard` in `React.memo` so it only re-renders when its specific player's props change
- B Move all player state into a single `useReducer` in the parent component
- C Use `useLayoutEffect` instead of `useEffect` for score updates
- D Convert `BatsmanCard` to a class component

Explanation: `React.memo` prevents re-renders when props are shallowly equal. If each `BatsmanCard` receives only its own player's data as props, wrapping it in `React.memo` means it only re-renders when that specific player's data changes. The other 10 players' cards skip re-rendering because their props (a different player object) didn't change.

Q: You're building the Dream11 fantasy team picker. It has three pieces of state: the current live match (updates every ball), the user's selected squad (updates on player pick), and the global leaderboard (updates every 5 minutes). Which state management setup best fits?

- A All three in a single `React Context` at the app root
- B** `React Query/TanStack Query` for server state (match, leaderboard) + local `useState` for squad selection
- C `Redux Toolkit` for all three, since it handles all state types uniformly
- D `Zustand` for all three to avoid prop drilling

Explanation: Match data and leaderboard are server state — they live on the server and need polling/websocket sync, staleness handling, and caching. `React Query` is purpose-built for this. Squad selection is pure client state local to the user session — `useState` or `Zustand` is appropriate. Mixing server and client state in a single `Context` or `Redux` store creates unnecessary complexity.

Q: In your run-chase calculator, `useEffect(() => { const needed = target - currentScore; setMessage(needed + ' runs from ' + overs + ' overs'); }, [target, currentScore])`. The `overs` variable comes from `useState` but is NOT in the `deps` array. What happens when `overs` changes?

- A** React throws a warning but the effect still runs with the new `overs` value
- B** The effect re-runs because React tracks all state variables automatically
- C** The effect uses a stale closure — it captures the `overs` value from when the effect was last registered, not the current value
- D** The component unmounts and remounts to clear the stale closure

Explanation: JavaScript closures capture variables by reference at the time the function is created. Since `overs` is not in the `deps` array, the effect function is only re-created when `target` or `currentScore` changes. In between, it holds a stale reference to the `overs` value from the last time the effect was registered. Always include all variables used inside `useEffect` in the `deps` array.

Q: You want a live ball-by-ball commentary feed (500+ items, updates every 30s from server) in a Server Component, with a 'Like' button on each commentary item that updates client-side. What is the correct App Router approach?

- A** Make the entire commentary page a Client Component with 'use client' so the Like button works
- B** Create a `CommentaryFeed` Server Component that fetches and renders the list, with a separate `LikeButton` Client Component nested inside each item
- C** Use a `useEffect` in a Client Component to fetch commentary, avoiding Server Components entirely for interactive pages
- D** Server Components cannot contain interactive elements, so this pattern is impossible

Explanation: Server Components can render static/fetched content efficiently (no JS bundle for the list). Client Components (with 'use client') can be nested inside Server Components as leaf nodes for interactivity. The `CommentaryFeed` fetches data server-side (fast, cacheable), and `LikeButton` is a small Client Component with its own state. This minimises the JavaScript sent to the browser while keeping interactivity.

KEY POINTS

- React.memo uses shallow reference equality — unstable function props defeat memoisation unless wrapped in useCallback.
- React 18 automatic batching applies to all state updates including async callbacks, reducing re-renders globally.
- useEffect cleanup runs before the next effect on dependency change, not only on unmount — critical for subscriptions.
- Server state (API data) belongs in TanStack Query; client UI state belongs in useState or Zustand — mixing them creates unnecessary complexity.
- Stale closures in useEffect occur when variables used inside the effect are missing from the dependency array.
- Server Components and Client Components compose — Server for data fetching, Client ('use client') as leaf nodes for interactivity.

System Design: Deep Dive & Advanced Patterns

Topic: System Design

Overview

System design is the discipline of creating distributed architectures that handle massive scale, failure, and concurrency. The problems it solves: how do you serve millions of users simultaneously without a single server crashing? How do you keep data consistent across geographically distributed databases? How do you recover from partial failures without the whole system going down? System design teaches the trade-offs between consistency, availability, and partition tolerance — choices that determine whether your system survives real-world conditions. It blends data modelling, caching, messaging, replication, and failure handling into coherent architectures that meet latency and reliability targets under unpredictable load.

🔧 Think of it like cricket: Designing a system at scale is like organising a World Cup — you can't run 10 simultaneous matches from one ground. You need distributed venues (distributed servers), clear rules about what happens if rain stops play (fault tolerance), a central scoreboard everyone trusts (consistency), and the ability to continue even if one stadium loses power (partition tolerance). The CAP theorem is cricket's playing conditions — you must choose which guarantees hold when things go wrong.

Core Concepts — The CAP Theorem

The CAP theorem states that a distributed system can guarantee at most two of three properties simultaneously: Consistency (every read returns the most recent write or an error), Availability (every request receives a response, though it may be stale), and Partition Tolerance (the system continues operating despite network partitions). In practice, network partitions always occur in distributed systems, so you must choose between CP (consistent but may be unavailable) and AP (available but may return stale data). CP systems include ZooKeeper, HBase, and relational databases configured for strong consistency. AP systems include Cassandra, CouchDB, and DNS. The choice depends on the business requirement: banking requires CP because you cannot show stale balances, whereas a social media feed can accept AP because a slightly stale feed is acceptable to users. The decision is not global; different subsystems can make different CAP trade-offs.

✂ Think of it like cricket: Imagine Cricinfo's live score system during the India vs Pakistan World Cup final. CP choice: every user sees the exact same score, but if a network partition occurs between the database and the web servers, the site goes down (unavailable). AP choice: the site stays up even during a partition, but some users might see a score that's 2 balls behind. Cricket fans prefer AP — a slightly stale score is better than Cricinfo crashing (as it famously did in 2011!). But for DRS ball-tracking systems that determine whether a batsman is out, you'd choose CP — the umpire decision must be consistent everywhere.

text

```
Mobile/Web Clients
  |
  v
[CDN: CloudFront] -- AP: serves cached scores, may be 5s stale
  |
  v
[API Gateway]
  |
  v
[Score Update Service] -- writes each ball event to Kafka
  |
  v
[Kafka Message Queue] -- durability: every ball delivery event is durable
  |
  +-----+
  v               v
[Redis Cache]      [PostgreSQL Primary]
AP, TTL=5s         CP: consistent, ~99.9% availability only
always responds,   |
may be 1-2 balls stale v
                   [Read Replicas]
  |
  v
[WebSocket Push Service] → Mobile Clients (live ball-by-ball)
```

Walking through the layers: the CDN serves cached scores for pure reads (AP, cheap, globally close to users). The Kafka queue decouples score ingestion from storage, so if PostgreSQL is momentarily slow, Kafka buffers the ball events durably. Redis provides sub-millisecond read latency for the current score (AP, with acceptable staleness for display). PostgreSQL stores the authoritative record used for historical and analytical queries. The WebSocket service pushes live updates to mobile clients. The CP/AP choice is made per layer based on exactly what staleness means for that specific use case.

How It Works Under the Hood — Consistent Hashing

When you have 10 IPL matches running simultaneously, you need to distribute their data across multiple database shards to avoid a single-node bottleneck. Naive modulo sharding ($\text{match_id} \% \text{num_shards}$) requires reshuffling almost all data when you add a shard — catastrophic at scale. Consistent hashing places both data keys and server nodes on a circular hash ring. Each key is assigned to the nearest node clockwise on the ring. When a node is added, only the keys between the new node and its predecessor move — about $1/N$ of total keys for N nodes — rather than the whole dataset. Virtual nodes (vnodes) distribute load evenly by mapping each physical server to many positions on the ring, smoothing out hotspots that would otherwise arise from uneven hashing. This is how Cassandra, DynamoDB, and Amazon's Dynamo paper implement data distribution at scale.

✏️ Think of it like cricket: Consistent hashing is like allocating cricket grounds to matches in a tournament. Instead of saying 'odd-numbered matches go to Ground A, even to Ground B' (modulo — breaks when you add a Ground C), you place all grounds on a circular schedule. Each match is assigned to the nearest available ground clockwise. When you add a new ground, only the matches that were scheduled at the ground immediately anti-clockwise shift — not every match in the tournament.

python

```
import hashlib
import bisect

class ConsistentHashRing:
    def __init__(self, replicas: int = 150):
        self.replicas = replicas
        self._ring = {}          # hash position → node
        self._sorted_keys = []   # sorted ring positions

    def _hash(self, key: str) → int:
        return int(hashlib.md5(key.encode()).hexdigest(), 16)

    def add_node(self, node: str):
        for i in range(self.replicas):
            h = self._hash(f"{node}#{i}")
            self._ring[h] = node
            bisect.insort(self._sorted_keys, h)

    def remove_node(self, node: str):
        for i in range(self.replicas):
            h = self._hash(f"{node}#{i}")
            if h in self._ring:
                del self._ring[h]
                self._sorted_keys.remove(h)

    def get_node(self, key: str) → str:
        if not self._ring:
            return None
        h = self._hash(key)
        idx = bisect.bisect(self._sorted_keys, h) % len(self._sorted_keys)
        return self._ring[self._sorted_keys[idx]]

ring = ConsistentHashRing()
for db in ["db-1", "db-2", "db-3"]:
    ring.add_node(db)

matches = ["ind_vs_aus", "eng_vs_sa", "pak_vs_nz", "ind_vs_pak", "aus_vs_eng"]
```



```
for m in matches:
    print(f"{m} → {ring.get_node(m)}")
```

💡 Pro Tip: For read-heavy workloads like cricket score queries (reads >> writes, ratio often 1000:1 during a match), use write-through caching: write to database AND cache simultaneously. Then use Redis PUBLISH on each ball event and WebSocket subscribers receive push updates — eliminating polling entirely and reducing database read load by 99%.

Common Patterns & Best Practices — CQRS & Circuit Breaker

CQRS (Command Query Responsibility Segregation) separates write operations (commands: 'record this ball') from read operations (queries: 'fetch the scorecard'). Commands go to an event store — an append-only log of BallDelivered events — while queries read from a denormalised projection optimised for display. Event Sourcing stores every state change as an immutable event, so the scorecard is derived by replaying events rather than overwriting state. The Circuit Breaker pattern prevents cascade failures: when calls to a downstream service (such as a Cricinfo data provider) start failing, the circuit 'opens' and fails fast for a timeout period instead of queuing requests that will all fail anyway. This protects the rest of the system from being dragged down by a slow or broken dependency.

🔪 Think of it like cricket: CQRS is like separating the scorer's role from the scoreboard operator. The scorer (command side) records every ball in the official scorebook (event log) — strictly sequential, never erased. The scoreboard operator (query side) reads the scorebook and updates the big display with a denormalised summary (projected read model). The circuit breaker is the Decision Review System (DRS) — if ball-tracking cameras keep failing (downstream service errors), the umpire doesn't keep waiting; they make a decision without DRS (fail fast) and review the system during the drinks break (circuit reset timeout).

python

```
import time
from enum import Enum

class CircuitState(Enum):
    CLOSED = "CLOSED"
    OPEN = "OPEN"
    HALF_OPEN = "HALF_OPEN"

class CircuitOpenError(Exception):
    pass

class CircuitBreaker:
    def __init__(self, threshold: int = 5, timeout: int = 30):
        self.failure_threshold = threshold
        self.timeout_seconds = timeout
        self.failure_count = 0
        self.state = CircuitState.CLOSED
        self.last_failure_time = 0.0

    def call(self, func, *args, **kwargs):
        if self.state == CircuitState.OPEN:
            if time.time() - self.last_failure_time ≥ self.timeout_seconds:
                self.state = CircuitState.HALF_OPEN
            else:
                raise CircuitOpenError("Circuit is OPEN – failing fast")

        try:
            result = func(*args, **kwargs)
        except Exception:
            self.failure_count += 1
            self.last_failure_time = time.time()
            if self.failure_count ≥ self.failure_threshold:
                self.state = CircuitState.OPEN
            raise
        else:
            self.failure_count = 0
            self.state = CircuitState.CLOSED
            return result
```

```
def fetch_live_score(match_id: str):
    # Calls the Cricinfo data provider...
    return {"match_id": match_id, "score": "245/4"}

cricinfo_breaker = CircuitBreaker(threshold=5, timeout=30)
result = cricinfo_breaker.call(fetch_live_score, match_id="ind_vs_pak_wc")
print(result)
```

⚠ Warning: Do NOT model cricket events as mutable database rows with UPDATE statements. When you UPDATE India's score from 245/4 to 246/4, you lose the history of every ball. Instead, model each delivery as an immutable INSERT (a BallDelivered event with over, ball, runs, extras, and wicket info). The scorecard becomes a materialised view derived from events — giving you full replay, an audit trail, and the ability to answer 'what was the score after 30 overs?' without any extra infrastructure.

Real-World Application

Hotstar served 25.3M concurrent viewers for the 2023 IPL final — the largest streaming event in history. They use Kafka for event streaming (each ball is a Kafka message), Redis clusters for live score caching, and horizontal scaling via Kubernetes. Dream11 uses consistent hashing to distribute fantasy point calculations across shards. Cricinfo's read architecture relies on CDN edge caching with a 5-second TTL for score updates. These are the exact patterns taught in this lesson, operating at production scale under enormous, spiky load.

KEY POINTS

- CAP theorem requires choosing CP (consistent but may be unavailable) or AP (available but may be stale) — the choice is per-layer, not per-system.
- Consistent hashing moves only 1/N of keys when adding a shard, versus 100% reshuffling for modulo-based partitioning.
- CQRS + Event Sourcing separates the write model (immutable event log) from the read model (denormalised projection) — each optimised independently.
- The circuit breaker has three states: CLOSED (normal), OPEN (failing fast), and HALF_OPEN (testing recovery) — preventing cascade failures.
- Write-through cache + Redis PUBLISH eliminates polling: each write updates both DB and cache and pushes to all WebSocket subscribers simultaneously.
- Virtual nodes in consistent hashing distribute load evenly by giving each physical server 150 ring positions — without them, uneven key distribution causes hotspots.

Q: Your cricket live score database suffers a network partition. Your system is configured as CP. A user in Mumbai requests the current score. What happens?

- A The user sees the last cached score (possibly stale)
- B The user gets an error or timeout — CP systems sacrifice availability to maintain consistency during partitions**
- C The system automatically switches to AP mode during partitions
- D The user's request is queued until the partition heals

Explanation: CP systems prioritise consistency over availability. During a partition, a CP system cannot guarantee that the response reflects the latest data across all nodes, so it refuses to respond (returns an error or timeout) rather than risk returning inconsistent data. This is the correct behaviour for a banking system but often too strict for sports scores.

Q: Your cricket live score API handles 2M req/s during peak. Scores update every 30 seconds. Which caching strategy minimises database load while keeping data fresh?

- A Cache-aside (lazy loading): fetch from DB on cache miss, set TTL=30s
- B Write-through with TTL=30s: every score update writes to cache AND database simultaneously**
- C No cache — use read replicas only to distribute database load
- D CDN caching only with TTL=5s at the edge

Explanation: Write-through ensures the cache always has fresh data immediately after each score update — no cache misses on the first read after an update. With TTL=30s matching the update frequency, data is never more than 30s stale. Cache-aside suffers a thundering herd problem: after TTL expiry at peak, thousands of requests simultaneously miss the cache and hit the database.

AWS Solutions Architect: Capstone Mini Project

Topic: AWS Solutions Architect

Project Overview

You will build a serverless live cricket score API on AWS that can serve your local cricket club, school tournament, or corporate cricket league. The system accepts ball-by-ball score updates via an API, stores them in DynamoDB, pushes wicket notifications to subscribers via SNS, and serves a static scoreboard dashboard from S3+CloudFront. It uses AWS Lambda, API Gateway, DynamoDB, SNS, S3, and CloudFront — the core AWS services for serverless applications. The architecture handles IPL-scale traffic spikes (0 to 2M requests in minutes) automatically via serverless autoscaling, with no servers to provision or patch. The entire system runs within the AWS Free Tier for small tournaments, making it both production-grade and inexpensive to operate.

🔧 Think of it like cricket: This project is the digital infrastructure for a cricket ground. API Gateway is the main entrance — it routes different types of visitors (scorers, viewers, umpires) to the right parts of the ground. Lambda functions are the specialists at each gate — the scorer's booth (`record_ball`), the scoreboard operator (`get_scorecard`), the match listing clerk (`list_matches`). DynamoDB is the official scorebook — immutable, durable records. SNS is the stadium PA system — when a wicket falls, it announces it to everyone subscribed (SMS, email, mobile push).

Learning Objectives

KEY POINTS

- Design a DynamoDB single-table schema with partition/sort key patterns and a GSI for efficient queries.
- Write Lambda functions in Python with boto3 for DynamoDB and SNS operations.
- Configure API Gateway with CORS, request validation, and usage plan throttling.
- Implement IAM least-privilege roles — each Lambda has only the permissions it needs.
- Deploy infrastructure as code using AWS SAM (CloudFormation).
- Set up CloudWatch metric alarms and X-Ray tracing for observability.
- Optimise Lambda cold starts with provisioned concurrency and function packaging.

Technical Requirements

KEY POINTS

- POST /matches — create a new match with team names, format (T20/ODI/Test), and venue.
- PUT /matches/{matchId}/ball — record a ball delivery with all delivery details.
- GET /matches/{matchId}/scorecard — return the full scorecard in <100ms p99 latency.
- GET /matches — list recent matches sorted by start time (via a DynamoDB GSI).
- SNS notification triggered automatically when wicket=True in a ball delivery.
- DynamoDB TTL: match data auto-deleted after 90 days to control storage costs.
- API Gateway usage plan: 1000 req/s burst, 500 req/s steady to prevent abuse.
- p99 latency < 100ms for GET scorecard — requires DynamoDB read optimisation.

Architecture & Design

Single-table DynamoDB design: PK=MATCH#{matchId}, SK=BALL#{over}#{ball} stores each delivery, while SK=META stores the match metadata (teams, format, venue, running totals). A Global Secondary Index (GSI) with PK=STATUS (active/completed) and SK=START_TIME enables chronological match listing without a table scan. Four Lambda functions back the API: create_match (POST /matches), record_ball (PUT /matches/{id}/ball), get_scorecard (GET /matches/{id}/scorecard), and list_matches (GET /matches). API Gateway fronts all Lambdas with CORS enabled. An SNS topic 'CricketEvents' receives wicket events and fans them out to email and SMS subscribers. An S3 bucket hosts the static scoreboard HTML/JS, and CloudFront caches it globally at the edge. For IAM, each Lambda has its own role granting only the specific DynamoDB actions it needs, following the principle of least privilege.

🔪 Think of it like cricket: The DynamoDB single-table design is like a well-organised physical scorebook — all match data in one book (one table), but with clear page organisation. The match metadata page (SK=META) is the match card at the front. Each ball's page (SK=BALL#14.3) is a delivery notation. The GSI is the tournament index at the back — listing all matches by date without flipping through every page.

yaml

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: Serverless live cricket score API

Globals:
  Function:
    Runtime: python3.11
    Timeout: 10
    MemorySize: 256
    Environment:
      Variables:
        TABLE_NAME: !Ref CricketMatchesTable
        SNS_TOPIC_ARN: !Ref CricketEventsSnsTopic

Resources:
  CricketMatchesTable:
    Type: AWS::DynamoDB::Table
    Properties:
      TableName: CricketMatches
      BillingMode: PAY_PER_REQUEST
      AttributeDefinitions:
        - AttributeName: PK
          AttributeType: S
        - AttributeName: SK
          AttributeType: S
        - AttributeName: status
          AttributeType: S
        - AttributeName: start_time
          AttributeType: S
      KeySchema:
        - AttributeName: PK
          KeyType: HASH
        - AttributeName: SK
          KeyType: RANGE
      GlobalSecondaryIndexes:
        - IndexName: StatusByTimeIndex
          KeySchema:
            - AttributeName: status
              KeyType: HASH
            - AttributeName: start_time
```

```

        KeyType: RANGE
        Projection:
          ProjectionType: ALL
      TimeToLiveSpecification:
        AttributeName: expiry_time
        Enabled: true

CricketEventsSnsTopic:
  Type: AWS::SNS::Topic
  Properties:
    TopicName: CricketEvents

CreateMatchFunction:
  Type: AWS::Serverless::Function
  Properties:
    Handler: handlers.create_match
    CodeUri: src/
    Policies:
      - DynamoDBCrudPolicy:
          TableName: !Ref CricketMatchesTable
    Events:
      CreateMatch:
        Type: Api
        Properties:
          Path: /matches
          Method: post
          RestApiId: !Ref CricketApi

RecordBallFunction:
  Type: AWS::Serverless::Function
  Properties:
    Handler: handlers.record_ball
    CodeUri: src/
    Policies:
      - Statement:
          - Effect: Allow
            Action:
              - dynamodb:PutItem
              - dynamodb:UpdateItem
            Resource: !GetAtt CricketMatchesTable.Arn
          - Effect: Allow
            Action:

```



```

      - sns:Publish
      Resource: !Ref CricketEventsSnsTopic
Events:
  RecordBall:
    Type: Api
    Properties:
      Path: /matches/{matchId}/ball
      Method: put
      RestApiId: !Ref CricketApi

GetScorecardFunction:
  Type: AWS::Serverless::Function
  Properties:
    Handler: handlers.get_scorecard
    CodeUri: src/
    Policies:
      - Statement:
          - Effect: Allow
            Action:
              - dynamodb:Query
            Resource: !GetAtt CricketMatchesTable.Arn
Events:
  GetScorecard:
    Type: Api
    Properties:
      Path: /matches/{matchId}/scorecard
      Method: get
      RestApiId: !Ref CricketApi

ListMatchesFunction:
  Type: AWS::Serverless::Function
  Properties:
    Handler: handlers.list_matches
    CodeUri: src/
    Policies:
      - Statement:
          - Effect: Allow
            Action:
              - dynamodb:Query
            Resource: !Sub "${CricketMatchesTable.Arn}/index/StatusByTimeIndex"
Events:
  ListMatches:

```

```
Type: Api
Properties:
  Path: /matches
  Method: get
  RestApiId: !Ref CricketApi
```

CricketApi:

```
Type: AWS::Serverless::Api
Properties:
  StageName: prod
  Cors:
    AllowMethods: "'GET,POST,PUT,OPTIONS'"
    AllowHeaders: "'Content-Type'"
    AllowOrigin: "'*'"
```

ScoreboardBucket:

```
Type: AWS::S3::Bucket
Properties:
  BucketName: cricket-scoreboard-dashboard
  WebsiteConfiguration:
    IndexDocument: index.html
    ErrorDocument: error.html
```

ScoreboardDistribution:

```
Type: AWS::CloudFront::Distribution
Properties:
  DistributionConfig:
    Enabled: true
    DefaultRootObject: index.html
    Origins:
      - Id: ScoreboardS3Origin
        DomainName: !GetAtt ScoreboardBucket.RegionalDomainName
        S3OriginConfig: {}
    DefaultCacheBehavior:
      TargetOriginId: ScoreboardS3Origin
      ViewerProtocolPolicy: redirect-to-https
      ForwardedValues:
        QueryString: false
```

Outputs:

```
ApiUrl:
  Description: Base URL of the Cricket Score API
```

```
Value: !Sub "https://${CricketApi}.execute-api.${AWS::Region}.amazonaws.com/prod"
SnsTopicArn:
Description: ARN of the CricketEvents SNS topic
Value: !Ref CricketEventsSnsTopic
```

Phase 1 — Core Implementation

Phase 1 implements match creation and ball recording. The key design decision is how to structure the DynamoDB items — using `SK=BALL#{over}#{ball}` allows querying all balls of a match with `PK=MATCH#{id}` and `SK begins_with('BALL#')`. The `UpdateExpression` on the `META` item maintains a running scorecard aggregate atomically — totals are incremented in place on each ball, so there's no need for a separate aggregation job and no race condition under concurrent writes.

 Think of it like cricket: A DynamoDB `UpdateExpression` is like the real-time scoreboard at the ground — it updates the total runs and wickets display instantly when each ball is recorded, without recounting every ball from the start. The ball delivery items (`SK=BALL#14.3`) are the detailed scorebook entries; the `META` item is the scoreboard summary.

python

```
import json, os, uuid, datetime
import boto3

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table(os.environ['TABLE_NAME'])
sns = boto3.client('sns')

def create_match(event, context):
    body = json.loads(event['body'])
    match_id = str(uuid.uuid4())[:8]
    now = datetime.datetime.utcnow()
    table.put_item(Item={
        'PK': f'MATCH#{match_id}',
        'SK': 'META',
        'team1': body['team1'],
        'team2': body['team2'],
        'format': body.get('format', 'T20'),
        'venue': body.get('venue', 'TBD'),
        'total_runs': 0,
        'total_wickets': 0,
        'balls_bowled': 0,
        'status': 'active',
        'start_time': now.isoformat(),
        'expiry_time': int((now + datetime.timedelta(days=90)).timestamp()),
    })
    return {'statusCode': 201, 'body': json.dumps({'matchId': match_id})}

def record_ball(event, context):
    match_id = event['pathParameters']['matchId']
    body = json.loads(event['body'])
    over = body['over']
    ball = body['ball']
    runs = body['runs']
    is_wicket = body.get('is_wicket', False)
    batsman = body.get('batsman', 'Unknown')
    bowler = body.get('bowler', 'Unknown')

    table.put_item(Item={
```

```

        'PK': f'MATCH#{match_id}',
        'SK': f'BALL#{over:02d}#{ball}',
        'over': over,
        'ball': ball,
        'runs': runs,
        'is_wicket': is_wicket,
        'batsman': batsman,
        'bowler': bowler,
        'extras': body.get('extras', 0),
        'delivery_type': body.get('delivery_type', 'legal'),
    })

    table.update_item(
        Key={'PK': f'MATCH#{match_id}', 'SK': 'META'},
        UpdateExpression='SET total_runs = total_runs + :r, '
                          'total_wickets = total_wickets + :w, '
                          'balls_bowled = balls_bowled + :b',
        ExpressionAttributeValues={
            ':r': runs,
            ':w': 1 if is_wicket else 0,
            ':b': 1,
        },
    )

    if is_wicket:
        sns.publish(
            TopicArn=os.environ['SNS_TOPIC_ARN'],
            Message=f'WICKET! {batsman} is out! Match {match_id} - '
                    f'{over}.{ball} overs. Bowler: {bowler}',
            Subject=f'Wicket Alert - Match {match_id}',
        )

    return {'statusCode': 200, 'body': json.dumps({'recorded': True, 'isWicket': is_wicket})}

```

Phase 2 — Feature Completion

Phase 2 adds the scorecard GET endpoint, which aggregates batting and bowling figures from all BALL# items for display, plus the match listing endpoint, which uses the GSI to return matches sorted by start time. It also adds response formatting so the scorecard data matches the structure the frontend expects. The GSI query avoids a full table scan when listing matches, keeping the operation fast and cheap even as the number of matches grows.

✂ Think of it like cricket: `get_scorecard` is like the scorer totalling up the scorebook at the end of an innings — querying all the individual ball entries (SK begins_with BALL#) and aggregating them into batting and bowling summaries. `list_matches` using the GSI is like the tournament secretary consulting the fixtures index rather than reading every scorebook to find today's matches.

python

```
import json
import boto3
from boto3.dynamodb.conditions import Key

def get_scorecard(event, context):
    match_id = event['pathParameters']['matchId']
    response = table.query(KeyConditionExpression=Key('PK').eq(f'MATCH#{match_id}'))
    items = response['Items']

    meta = next((i for i in items if i['SK'] == 'META'), None)
    if not meta:
        return {'statusCode': 404, 'body': json.dumps({'error': 'Match not found'})}

    balls = [i for i in items if i['SK'].startswith('BALL#')]
    batting = {}
    bowling = {}
    for ball in balls:
        b = ball['batsman']
        bw = ball['bowler']
        batting.setdefault(b, {'runs': 0, 'balls': 0, 'fours': 0, 'sixes': 0})
        batting[b]['runs'] += ball['runs']
        batting[b]['balls'] += 1
        if ball['runs'] == 4:
            batting[b]['fours'] += 1
        if ball['runs'] == 6:
            batting[b]['sixes'] += 1
        bowling.setdefault(bw, {'runs': 0, 'balls': 0, 'wickets': 0})
        bowling[bw]['runs'] += ball['runs']
        bowling[bw]['balls'] += 1
        if ball.get('is_wicket'):
            bowling[bw]['wickets'] += 1

    scorecard = {
        'matchId': match_id,
        'team1': meta['team1'],
        'team2': meta['team2'],
        'totalRuns': int(meta['total_runs']),
        'totalWickets': int(meta['total_wickets']),
        'batting': [
```

```

        {
            'batsman': k,
            'runs': v['runs'],
            'balls': v['balls'],
            'strikeRate': round(v['runs'] / v['balls'] * 100, 1) if v['balls'] > 0 else 0,
        }
        for k, v in batting.items()
    ],
    'bowling': [
        {
            'bowler': k,
            'overs': round(v['balls'] / 6, 1),
            'runs': v['runs'],
            'wickets': v['wickets'],
        }
        for k, v in bowling.items()
    ],
}

return {
    'statusCode': 200,
    'headers': {'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*'},
    'body': json.dumps(scorecard),
}

```

```

def list_matches(event, context):
    response = table.query(
        IndexName='StatusByTimeIndex',
        KeyConditionExpression=Key('status').eq('active'),
        ScanIndexForward=False,
    )
    matches = [
        {'matchId': i['PK'].split('#')[1], 'team1': i['team1'],
        'team2': i['team2'], 'startTime': i['start_time']}
        for i in response['Items']
    ]
    return {
        'statusCode': 200,
        'headers': {'Content-Type': 'application/json', 'Access-Control-Allow-Origin': '*'},
        'body': json.dumps({'matches': matches}),
    }

```


Phase 3 — Polish & Production Readiness

Production hardening covers input validation (delivery type must be 'legal', 'wide', or 'no-ball'; runs 0-6; the match must exist), structured logging for CloudWatch Insights queries, X-Ray tracing to identify slow DynamoDB calls, a Dead Letter Queue (DLQ) for failed SNS deliveries, and a CloudWatch alarm when the error rate exceeds 1%. These additions are what separate a demo from a system you can trust during a live match with real users watching.

 Think of it like cricket: Production hardening is the third umpire and match officials behind the scenes. Input validation is the boundary rope check — an illegal delivery (a no-ball outside the rules) is rejected before it counts. Structured logging and X-Ray tracing are the match replay cameras that let you review exactly what happened on any delivery. The CloudWatch alarm is the match referee who steps in the moment something goes wrong on the field.

```
python
```

```
import json
import logging
from pydantic import BaseModel, validator, Field
import aws_xray_sdk.core as xray

xray.patch_all()

class BallDelivery(BaseModel):
    over: int = Field(..., ge=0, le=49)
    ball: int = Field(..., ge=1, le=6)
    runs: int = Field(..., ge=0, le=6)
    batsman: str = Field(..., min_length=2, max_length=50)
    bowler: str = Field(..., min_length=2, max_length=50)
    is_wicket: bool = False
    delivery_type: str = 'legal'

    @validator('delivery_type')
    def validate_delivery_type(cls, v):
        assert v in ('legal', 'wide', 'no-ball', 'bye', 'leg-bye'), \
            f"Invalid delivery type: {v}"
        return v

def lambda_handler_with_logging(event, context):
    logger = logging.getLogger()
    logger.setLevel(logging.INFO)
    match_id = event.get('pathParameters', {}).get('matchId')
    logger.info(json.dumps({
        'event': 'ball_record',
        'matchId': match_id,
        'requestId': context.aws_request_id,
    }))

    try:
        body = BallDelivery(**json.loads(event['body']))
    except Exception as e:
        return {'statusCode': 422,
            'body': json.dumps({'error': f'Validation failed: {str(e)}'})}
```

```
# ... proceed with the validated body (record ball, update META, publish SNS) ...
return {'statusCode': 200, 'body': json.dumps({'recorded': True, 'matchId': match_id})}
```

Evaluation Rubric

KEY POINTS

- DynamoDB Design: PK/SK pattern enables all 4 access patterns without scans; the GSI correctly supports chronological match listing.
- IAM Least Privilege: each Lambda role has only the specific actions needed (ddb:PutItem for record_ball, not full ddb:*).
- Error Handling: 404 for a missing match, 422 for an invalid delivery, 500 with a structured error body for unexpected failures.
- Observability: CloudWatch alarm on error rate, X-Ray tracing enabled, structured JSON logs with matchId/requestId for filtering.
- Cost Awareness: DynamoDB PAY_PER_REQUEST + TTL for auto-cleanup + Lambda free tier = \$0 for small tournaments.
- Input Validation: a Pydantic model rejects invalid delivery types, out-of-range runs, and missing required fields at the API boundary.
- SNS Integration: wicket notifications trigger correctly, and a DLQ catches failed deliveries for retry.

💡 Extension Challenges: 1) Add an AWS WebSocket API Gateway so the scoreboard updates in real time without polling — each ball delivery triggers a Lambda that pushes the update to all connected WebSocket clients. 2) Use Lambda@Edge to serve the scoreboard in the viewer's local timezone (IST, GMT, AEST) based on CloudFront geolocation headers. 3) Build a Dream11-style fantasy points Lambda triggered by DynamoDB Streams — every ball event automatically calculates and updates fantasy points for all players in real time.